



# 【金九银十】大厂最新面试原题解析

## 1. 往期部分专题

- [📖 以终为始，重回前端 -- Web Development Trend](#)
- [📖 以终为始，重回前端 --Typescript： JavaScript with syntax for types](#)
- [📖 以终为始，重回前端 -- Awesome JavaScript & Design Patterns](#)
- [📖 以终为始，重回前端（特别篇） -- New React 19 Features You Should Know & How to Upgrade](#)
- [📖 以终为始，重回前端 -- 构建高性能Vue应用：揭秘 Module Federation 微前端架构的核心技术](#)
- [📖 以终为始，重回前端 -- 揭秘大厂工程化内核，从软件工程视角剖析编译时、运行时及打包构建三板斧](#)
- [📖 以终为始，重回前端 -- 字节前端专家揭秘大厂React最佳实践，手把手从0打造一个开源社区前沿的组件库](#)
- [📖 2025 金三银四面试突击早鸟课 -- 一线25K 前端P6岗的候选人能力要求 & 画像解析](#)
- [📖 2025 金三银四面试突击早鸟课第二弹——字节T2-1岗位面试真题解析](#)
- [📖 2025 金三银四面试突击早鸟课第三弹——手写对标一线城市25K前端P6岗的满分简历](#)
- [📖 金三银四面试突击ALL IN ONE 第三篇 —— 大厂P6级模拟面试来啦](#)
- [📖 金三银四面试突击ALL IN ONE 第四篇 —— 25年金三银四面试高频踩坑解析](#)
- [📖 金三银四面试突击ALL IN ONE 第五篇 —— 金三银四面试利器之前端技术架构设计实操](#)
- [📖 金三银四面试突击ALL IN ONE 第六篇 —— 金三银四大厂真实面试原题解析](#)
- [📖 618专题--大促搭投实践&高可用性保障](#)
- [📖 年中复盘会（1/2） —— 25年上半年中大厂高频面试总结](#)
- [📖 年中复盘会（2/2） —— 25年上半年中大厂高频面试总结](#)
- [📖 25年下半年面试真题预测及解析](#)
- [📖 阿里/字节面试高频真题解析与9月考点预测](#)
- [📖 阿里&字节9月面试高频真题预测](#)

## 2. 原题分析

## 2.1 Call apply bind new

### Call

Code block

```
1  Function.prototype.call2 = function(context,
2    // 判断是否是undefined和null
3    if (typeof context === 'undefined' ||
4    context === null) {
5      context = window
6    }
7    let fnSymbol = Symbol()
8    context[fnSymbol] = this
9    let fn = context[fnSymbol](...args)
10   delete context[fnSymbol]
11   return fn
12 }
```

### apply

Code block

```
1  Function.prototype.apply2 = function(context,
2    args) {
3    // 判断是否是undefined和null
4    if (typeof context === 'undefined' ||
5    context === null) {
6      context = window
7    }
8    let fnSymbol = Symbol()
9    context[fnSymbol] = this
10   let fn = context[fnSymbol](...args)
11   delete context[fnSymbol]
12   return fn
13 }
```

### Bind

Code block

```
1  Function.prototype.bind2 = function (context)
2  {
3    if (typeof this !== "function") {
4      throw new
5      Error("Function.prototype.bind - what is
6      trying to be bound is not callable");
7    }
8    var self = this;
9    var args =
10    Array.prototype.slice.call(arguments, 1);
11
12    var fNOP = function () {};
13
14    var fBound = function () {
15      var bindArgs =
16      Array.prototype.slice.call(arguments);
17      return self.apply(this instanceof
18      fNOP ? this : context, args.concat(bindArgs));
19    }
20
21    fNOP.prototype = this.prototype;
22    fBound.prototype = new fNOP();
23    return fBound;
24 }
```

### New

Code block

```
1  function objectFactory() {
2    var obj = new Object(),
3    Constructor = [].shift.call(arguments);
4    obj.__proto__ = Constructor.prototype;
5    var ret = Constructor.apply(obj,
6    arguments);
7    return typeof ret === 'object' ? ret :
8    obj;
9  }
```

## 2.2 BFC

Block formatting context 块级格式化上下文

指的是一个独立的区域，在这个区域内，块级盒子(block-level box) 按照一定的规则进行布局，并且这个区域的布局不受外部的影响。简单来说，BFC就像一个独立的容器，里面的元素布局

## 2.3 Margin 重叠

1. 两个正数的外边距取最大的边距作为外边距。
2. 如果一个为正数，一个为负数，最终的外边距为两者之和。
3. 如果两个值都是负数的话，最终的外边距取绝对值最大的值。

规则与其他区域无关，而这个容器的高度计算也会将浮动元素考虑在内

BFC 的主要作用和特点：

- **隔离布局:**BFC 内部的块级元素布局不受外部的影响，反之亦然。
  - **清除浮动:**可以用来解决浮动元素导致的父元素高度塌陷问题。
  - **避免外边距折叠:**属于同一个BFC 的相邻块级元素的垂直方向外边距会发生重叠，而不同BFC 之间的元素则不会。
  - **计算高度:**BFC 的高度计算会将浮动元素也包含在内，避免了浮动元素撑开父元素高度的问题。
  - **独立渲染区域:**BFC 是一个独立的渲染区域，它内部的元素布局规则与其他区域无关，这使得我们可以更好地控制页面的布局。
- `float` 除了 `none` 以外的值
  - `overflow` 除了 `visible` 以外的值
  - `display: inline-block`
  - `display: table-cell`
  - `display: flex` / `display: inline-flex`
  - `display: grid` / `display: inline-grid`
  - `position: absolute` / `position: fixed`
  - `contain: layout`

## 2.4 Let const var

1. `var` : 函数作用域 (function-scoped) `let` / `const` : 块级作用域 (block-scoped)
2. `var` : 变量会被提升到函数或全局作用域的顶部，初始值为 `undefined` ； `let` / `const` : 也会提升，但不会被初始化（暂时性死区 TDZ； *ReferenceError*）
3. `var` : 在全局作用域声明时，会成为全局对象(`window`)的属性； `let` / `const` : 不会成为全局对象的属性

Code block

```
1  let x = 1;
2  console.log(window.x); // undefined
```

## 2.5 Map

- 键可以是任意数据类型（对象、函数等）
- 保持插入顺序
- 高效的查找操作（接近 O(1) 时间复杂度）
- 可以直接遍历

`margin` 重叠问题指的是相邻元素之间的垂直 `margin` 值会合并成一个，而不是两个 `margin` 值简单相加。

1. 添加边框或内边距:

在元素之间添加边框或内边距可以阻止 `margin` 的合并，因为边框和内边距不会被重叠。

2. 使用 `overflow` 属性:

在父元素上设置 `overflow: hidden;` 或其他非 `visible` 值，可以阻止 `margin` 塌陷。

3. 使用Flexbox 或Grid 布局:

Flexbox 和Grid 布局在布局元素时，通常不会发生 `margin` 重叠。

4. 使用 `clear` 属性:

`clear` 属性主要用于清除浮动，但也可以用来阻止 `margin` 重叠，特别是在浮动元素和其他元素之间。

5. 调整 `margin` 值:

如果可能，可以调整元素的 `margin` 值，使其不会重叠。例如，如果两个元素之间需要20px 的间距，可以给其中一个元素设置 `margin-bottom: 20px;`，另一个元素设置 `margin-top: 0;`。

6. 使用负 `margin`:

在某些情况下，可以使用负 `margin` 来抵消重叠的 `margin`。

7. 使用BFC (Block Formatting Context):

创建BFC 也可以阻止 `margin` 重叠。例如，设置 `overflow: hidden;` 或 `display: flow-root;` 可以创建一个BFC。

## 2.6 WeakMap

1. 键必须是对象（不能是原始值）
2. 键是弱引用的（不会阻止垃圾回收）
3. 不可枚举（没有方法可以获取所有键或值）
4. 没有 size 属性

vue3 proxy 使用weakMap，用于处理原始对象不存在时

```
1  const map = new Map();
2
3  // 添加元素
4  map.set('name', 'Alice');
5  map.set(1, 'number one');
6
7  // 获取元素
8  map.get('name'); // 'Alice'
9
10 // 检查键是否存在
11 map.has('name'); // true
12
13 // 删除元素
14 map.delete('name');
15
16 // 大小
17 map.size;
18
19 // 遍历
20 map.forEach((value, key) => console.log(key, value));
21 for (const [key, value] of map) { /* ... */ }
```

## 使用场景

### 1. 需要键不是字符串的情况

Code block

```
1  const domNodes = new Map();
2  const element =
    document.querySelector('#someId');
3  domNodes.set(element, { clicks: 0 });
```

### 2. 需要维护插入顺序的键值对

Code block

```
1  const orderedData = new Map();
2  orderedData.set('first', 1);
3  orderedData.set('second', 2);
```

### 3. 频繁增删键值对的场景

Code block

```
1  const cache = new Map();
2  function getData(key) {
3    if (cache.has(key)) return cache.get(key);
4    const data = fetchData(key); // 假设的获取数据函数
5    cache.set(key, data);
6    return data;
7  }
```

1. 自动内存管理：当组件销毁时，相关响应式数据自动解除关联
2. 无额外清理负担：开发者无需手动清理响应式引用

Code block

```
1  const weakMap = new WeakMap();
2
3  const obj1 = {};
4  const obj2 = {};
5
6  // 设置键值对
7  weakMap.set(obj1, 'private data 1');
8  weakMap.set(obj2, 'private data 2');
9
10 // 获取值
11 console.log(weakMap.get(obj1)); // 'private data 1'
12
13 // 检查键是否存在
14 console.log(weakMap.has(obj1)); // true
15
16 // 删除键值对
17 weakMap.delete(obj1);
```

### 1. 弱引用特性

WeakMap 对键是弱引用，这意味着：

- 如果没有其他引用指向键对象，该对象会被垃圾回收
- 即使 WeakMap 中仍然引用该对象，也不会阻止垃圾回收

Code block

```
1  let obj = { id: 1 };
2  const weakMap = new WeakMap();
3  weakMap.set(obj, 'some data');
4
5  obj = null; // 现在 { id: 1 } 可以被垃圾回收，
               WeakMap 中的条目会自动移除
```

### 2. 不可迭代

WeakMap 没有以下方法：

- `keys()` `values()` `entries()` `forEach()` `size` 属性

### 3. 键必须是对象

Code block

```
1  const weakMap = new WeakMap();
2
3  weakMap.set({}, 'valid'); // 正确
4  weakMap.set('key', 'value'); // TypeError:
    Invalid value used as weak map key
```

## 使用场景

### 1. 缓存计算结果 当对象不存在时自动清除

```
1 const cache = new WeakMap();
2
3 function computeExpensiveValue(obj) {
4   if (cache.has(obj)) {
5     return cache.get(obj);
6   }
7
8   const result = /* 复杂计算 */;
9   cache.set(obj, result);
10  return result;
11 }
```

2. 存储 DOM 元素的附加数据（元素移除时自动清理）

```
Code block

1  const elementHandlers = new WeakMap();
2
3  function setupHandler(element) {
4    const handler = () =>
5      console.log('Clicked!');
6    element.addEventListener('click', handler);
7    elementHandlers.set(element, handler);
8  }
9
10 function teardownHandler(element) {
11   const handler =
12     elementHandlers.get(element);
13   element.removeEventListener('click',
14     handler);
15   elementHandlers.delete(element);
16 }
```

2.7 Set

```
Code block

1  const uniqueNames = new Set();
2  uniqueNames.add('Alice'); // 字符串
3  uniqueNames.add(42);      // 数字
4  uniqueNames.add({});      // 对象
```

需要遍历集合或知道集合大小

```
Code block

1  const colors = new Set(['red', 'green',
2    'blue']);
3  console.log(colors.size); // 3
4
5  for (const color of colors) {
6    console.log(color);
7  }
```

需要实现集合运算（并集、交集等）

```
Code block

1  const setA = new Set([1, 2, 3]);
2  const setB = new Set([2, 3, 4]);
```

2.8 WeakSet

核心区别对比

| 特性    | Set                      | WeakSet                |
|-------|--------------------------|------------------------|
| 值类型   | 任意类型（对象、原始值等）            | 只能是对象（非原始值）            |
| 垃圾回收  | 强引用（阻止值被回收）              | 弱引用（不阻止值被回收）           |
| 可迭代性  | 可遍历（keys/values/entries） | 不可遍历                   |
| 大小查询  | 有 <code>size</code> 属性   | 无 <code>size</code> 属性 |
| 方法完整性 | 完整API（add/delete/has等）   | 只有基本操作（add/delete/has） |
| 内存管理  | 可能导致内存泄漏                 | 自动清理无引用的值              |

```
3
4 // 并集
5 const union = new Set([...setA, ...setB]);
6
7 // 交集
8 const intersection = new
  Set([...setA].filter(x => setB.has(x)));
```

## 2.9 移动端1px

- transform缩放
- viewport + rem
- Box-shadow

Code block

```
1 .border-1px {
2   position: relative;
3 }
4 .border-1px::after {
5   content: "";
6   position: absolute;
7   left: 0;
8   bottom: 0;
9   width: 100%;
10  height: 1px;
11  background: #000;
12  transform: scaleY(0.5);
13  transform-origin: 0 0;
14 }
```

Code block

```
1 <meta name="viewport"
2 content="width=device-
width,initial-
scale=1.0,maximum-
scale=1.0,minimum-
scale=1.0,user-
scalable=no">
```

Code block

```
1 .border-1px {
2   border-bottom: 1px
solid #000;
3 }
```

- 通过 JS 对文档进行修改，所以性能上有一定影响
- 会对项目中所有使用 `rem` 单位的对象进行影响。如果是老项目，则会全部更改 css 样式（不适合老项目改造）

Code block

```
1 .border-1px {
2   box-shadow: 0 0.5px 0 0
#000;
3 }
```

## 2.10 后端传大数据如何减少渲染负担

### 1. 数据分页与懒加载

- 分页处理：只传输当前页面需要的数据
- 无限滚动：动态加载数据，用户滚动时再请求新数据 **intersection observer**
- 虚拟滚动：只渲染可视区域内的数据行 **scroll top**

### 2. 数据优化

- 压缩数据：使用Gzip等压缩算法减小传输体积

### 3. 前端渲染优化

- Web Workers：将数据处理移出主线程
- 分批渲染：将大数据分成小块逐步渲染

### 4. 缓存策略

- 本地存储：将不变的数据存储在localStorage或IndexedDB
- 服务端缓存：对频繁请求的数据进行缓存

### 5. 服务端优化

- 流式传输：使用SSE(Server-Sent Events)或WebSocket逐步发送数据
- GraphQL：让前端精确请求所需数据，避免过度获取

### 3. 9~10月如何平稳收获offer?



EncodeStudio  
印客学院

印客学院2025匠心之作  
内容&服务全面升级

# WEB

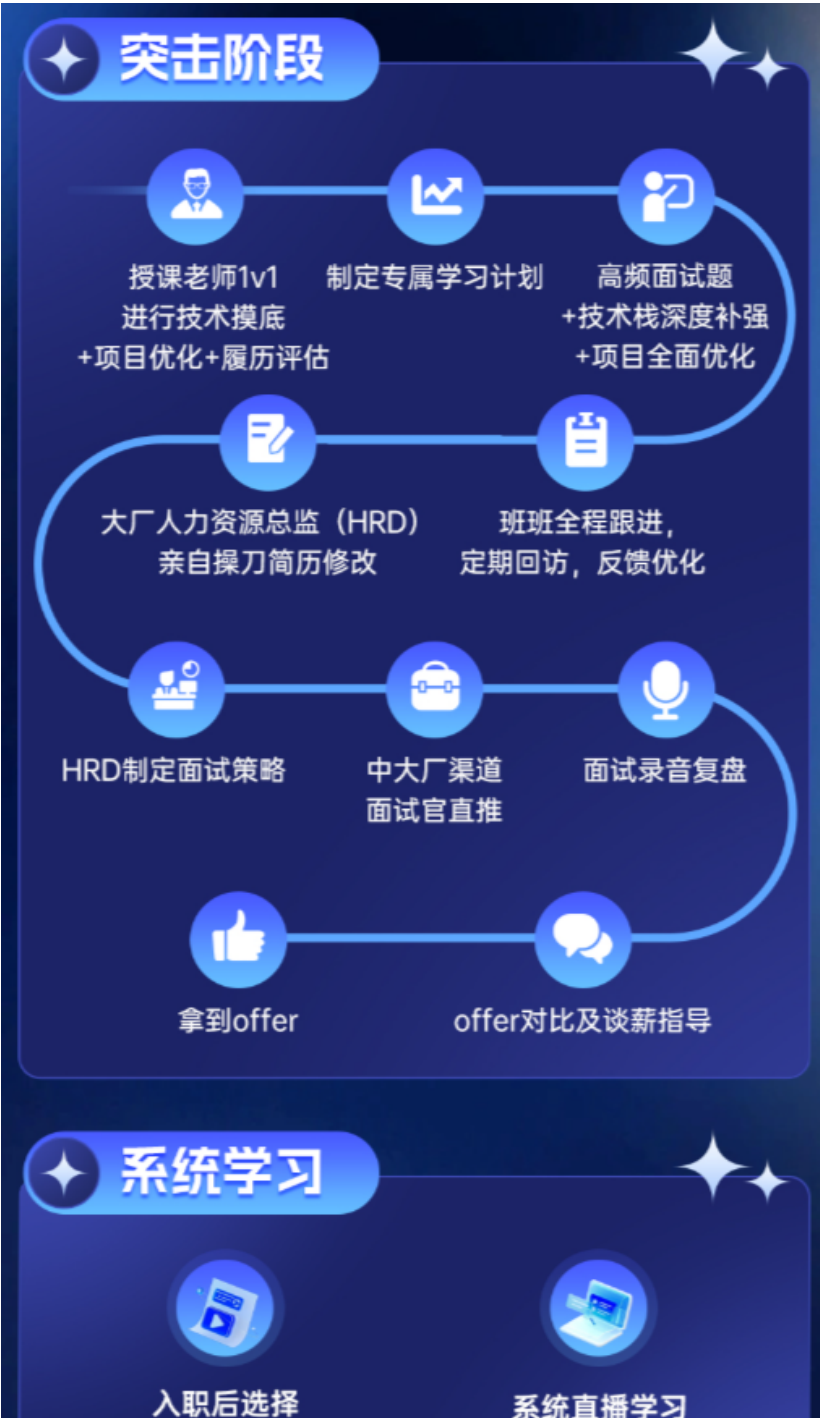
## Web前端

### 大厂工程师训练营

进阶前端架构  
直击阿里P7



全程定制，学习计划+简历指导+面试指导+突击指导+全国内推+试用期全程陪跑，全网独家2V1模式，双师辅导（现役P8+HRD）



3.1 专业一对一

近期一对一

- 📅 (2025-05-06) xxx-2.5年-React-24K
- 📅 (2025-05-07) xxx-1年-React-15k
- 📅 (2025-05-25) xxx-7年-Vue-20k

报名学员专享

📖 13. 如何写出高质量的简历（For 付费学员）

3.2 渠道内推 & 保障就业

渠道：面试官直推/高管直推（简历直达leader）

保障：合同保障最低薪资涨幅，保障心仪企业offer，无限次直推，直到满意为止

4. 真题预测

4.1 Require VS Import （CommonJS 和 ESM 区别）

4.1.1 Require

4.1.3 区别

`node` 模块的运行机制就是在每一个模块外层包裹了一层函数，有了函数的包裹就可以实现代码间的作用域隔离。

### 4.1.2 import

<https://juejin.cn/post/7098192216229117959#heading-22>

1. ES Module 输出的是值的引用，而 CommonJS 输出的是值的拷贝，一旦输出一个值，模块内部的变化就影响不到这个值；
2. ES Module 是编译时执行，而 CommonJS 模块是在运行时加载；
3. ES6 Module可以导出多个值，而CommonJs 是单个值导出；
4. ES6 Module 静态语法只能写在顶层，而CommonJs 是动态语法可以写在判断里；
5. ES6 Module的 this 是 undefined，而CommonJs 的 this 是当前模块；

Code block

```
1 // esm
2 // lib.js
3 export let counter = 3;
4 export function incCounter() {
5   counter++;
6 }
7 // main.js
8 import { counter, incCounter } from './lib';
9 console.log(counter); // 3
10 incCounter();
11 console.log(counter); // 4
```

## 4.2 webpack 相关

### 4.2.1 Webpack 完整流程

<https://juejin.cn/post/7158707111315111972#heading-2>

### 4.2.2 Webpack loader

loader执行原理：  
<https://juejin.cn/post/6992754161221632030#heading-0>  
常见loader：  
<https://juejin.cn/post/6942322281913778206>

### 4.2.3 Webpack plugin

- 1. HtmlWebpackPlugin: 生成HTML 文件.
- 2. CleanWebpackPlugin: 清理构建输出目录. 
- 3. CopyWebpackPlugin: 复制静态文件. 
- 4. MiniCssExtractPlugin: 提取CSS 文件. 
- 5. OptimizeCssAssetsPlugin: 优化和压缩CSS 文件. 
- 6. TerserPlugin: 压缩JavaScript 文件.
- 7. webpack.DefinePlugin: 定义全局变量.
- 8. PurgeCSSPlugin: 删除未使用的CSS 样式.
- 9. webpack.ProvidePlugin: 在模块中自动加载依赖.
- 10. BundleAnalyzerPlugin: 分析构建后的包.
- 11. ImageminPlugin: 优化图片. 
- 12. CompressionPlugin: 生成gzip 或brotli 文件.
- 13. webpack.NoEmitOnErrorsPlugin: 避免在错误时生成文件. 
- 14. webpack.HotModuleReplacementPlugin: 热模块替换. 
- 15. optimization.splitChunks: 代码分割. 

### 4.2.4 Webpack hash 的作用对比

<https://juejin.cn/post/7146489193491857421#heading-0>

fullhash：所有构建产物均使用相同的 hash。这意味着修改任何一个文件，所有文件的 hash 值都将会被改变

chunkhash：同一 chunk 下的文件使用相同的 hash。修改在同一个chunk中的文件，同属该 chunk 的所有文件 hash 值都会改变

contenthash：不同的文件使用不同的 hash。每个文件都会根据自己的文件内容生成 hash 值，互不干扰。

### 4.2.5 Module Federation

<https://juejin.cn/post/7151281452716392462#heading-5>

## 4.3 性能优化

定量指标：

1. Largest Contentful Paint : 最大内容绘制, 测量加载性能。理想值 < 2.5 秒。
2. First Input Delay : 首次输入延迟, 测量交互性。理想值 < 100 毫秒。
3. Cumulative Layout Shift : 累积布局偏移, 测量视觉稳定性。理想值 < 0.1。

### 4.3.1 编译构建时优化

1. **体积优化**: terser 压缩、混淆
2. **资源优化**
  - 图片: webp等、srcset 基于dpi尺寸
  - 字体: 预加载关键字体: `<link rel="preload" as="font">`。
  - 懒加载: script defer。
  - CDN
3. **减少 HTTP 请求**
  - **文件合并**: 在 HTTP/1.1 时代常用, 但在 HTTP/2+ 下需权衡 (多路复用减少了合并的必要性, 合并可能影响缓存粒度)。小图标合并成雪碧图仍有价值。
  - **内联关键 CSS/JS**: 首屏内容 & 骨架屏 所必需的小段 CSS/JS 直接内联在 HTML 中, 避免阻塞渲染的请求。
4. **利用浏览器缓存**
  - **强缓存 (Cache-Control, Expires)**: 为静态资源设置较长的过期时间 (e.g., `max-age=31536000`, `immutable`)。
  - **协商缓存 (ETag, Last-Modified)**: 确保用户代理能有效验证资源是否过期。
  - **Service Worker**: 实现更精细的缓存策略 (网络优先、缓存优先等), 支持离线访问。PWA 的核心。
5. **减少资源体积**
  - Code Splitting
  - Tree Shaking
  - Webpack Bundle Analyzer。
6. **预加载、预连接、预获取**
  - `<link rel="preload">` : \*\* 强制浏览器提前加载关键资源 (字体、首屏图片、关键 JS/CSS)。
  - `<link rel="preconnect">` : \*\* 提前建立与关键第三方源 (CDN, API 域名, 分析服务) 的连接 (DNS 查找、TCP 握手、TLS 协商)。
  - `<link rel="prefetch">` : \*\* 在浏览器空闲时预取用户下一步可能需要的资源 (如下一页的内容、非首屏图片)。
  - `<link rel="dns-prefetch">` : \*\* 提前解析域名 DNS (比 `preconnect` 更轻量)。
7. **HTTP/2 或 HTTP/3**

### 4.3.2 运行时优化

1. **优化 CSS**
  - a. 避免深层嵌套选择器: 降低匹配复杂度。
  - b. 减少过度使用昂贵的属性: 如 `box-shadow`, `border-radius`, `filter`
2. **优化 JavaScript 执行**
  - a. 避免长任务: 将耗时 JS 任务分解成小块 (使用 `setTimeout`, `requestAnimationFrame`, `requestIdleCallback`, Web Workers)。
  - b. 防抖和节流: 控制高频事件 (resize, scroll, input) 的处理频率。
  - c. Web Workers: 将复杂的计算任务移出主线程, 避免阻塞 UI 渲染和交互响应。
  - d. 优化事件处理: 使用事件委托, 及时移除不再需要的事件监听器。
  - e. 虚拟列表: 对于超长列表, 仅渲染可视区域内的项 (`react-window`, `vue-virtual-scroller`)。
3. **优化渲染流程**:
  - a. 减少重排和重绘: 集中修改 DOM 样式 (使用 `class` 切换), 避免逐个修改。离线操作 DOM (使用 `DocumentFragment`, `cloneNode`, 隐藏元素修改完再显示)。
  - b. GPU 加速: 对动画元素使用 `transform` 和 `opacity` 属性, 它们通常在合成层处理, 避免触发重排重绘。
4. **框架优化: 避免不必要的渲染**
  - a. React: 使用 `React.memo`, `PureComponent`, `useMemo`, `useCallback`。优化 Context 的使用。
  - b. Vue: 利用 `v-once`, `v-memo` (Vue 3), 计算属性缓存, 合理设计组件。

### 4.3.3 用户体验

1. 骨架屏
2. 占位符
3. 进度条
4. 交互优化 Hover click etc
5. 资源缓存 LRU

- 确保服务器和 CDN 支持 HTTP/2 (多路复用、头部压缩、服务器推送 - 需谨慎使用) 或更先进的 HTTP/3 (基于 QUIC，改进弱网性能)。

## 4.4 打开url访问完整流程

### 1. DNS 解析

浏览器需要将域名（如 `www.example.com`）转换为服务器的 IP 地址。这个过程称为 **\*\*DNS 解析\*\***：

- a. 浏览器首先检查本地缓存，看是否已经缓存了该域名的 IP 地址。
- b. 如果本地缓存中没有，浏览器会向 **ISP（互联网服务提供商）的 DNS 服务器** 发送 DNS 查询请求。
- c. 如果 ISP 的 DNS 服务器也没有缓存，它会向上级 DNS 服务器查询，直到找到根 DNS 服务器。
- d. 最终，DNS 解析返回目标服务器的 IP 地址。

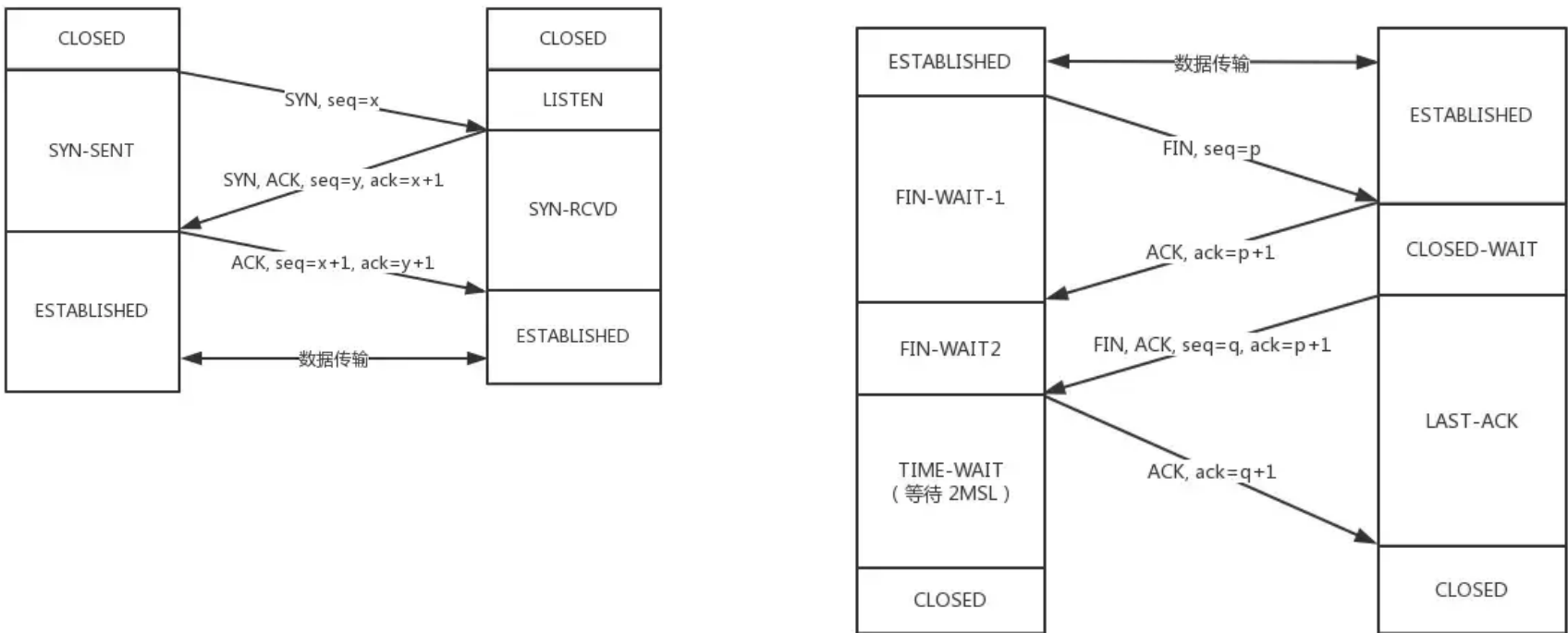
### 2. CDN

- a. DNS 服务器根据用户 IP 地址，将域名解析成相应节点的缓存服务器IP地址，实现用户就近访问

### 3. 建立网络连接

浏览器使用解析到的 IP 地址，与目标服务器建立网络连接：

- 1. **TCP 连接**：浏览器发起一个 TCP 三次握手过程，与服务器建立可靠的 TCP 连接。
- 2. **TLS/SSL 加密**：如果是 HTTPS 协议，浏览器和服务器会进行 TLS/SSL 握手，协商加密密钥，确保数据传输的安全。



### 4. 发送 HTTP/HTTPS 请求

浏览器通过已建立的连接，向服务器发送 HTTP 或 HTTPS 请求。请求包括：

- **请求行**：例如 `GET / HTTP/1.1`，表示请求根路径。
- **请求头**：包含浏览器信息（User-Agent）、语言偏好、请求来源（Referer）等。
- **请求体**（如果适用）：例如在 POST 请求中，包含表单数据或其他内容。

### 5. 服务器处理请求

服务器接收到请求后，根据请求内容进行处理：

### 6. 浏览器接收响应

服务器将响应通过网络发送回浏览器：

- 1. 浏览器接收完整的 HTTP 响应。
- 2. 浏览器解析响应头，根据 `Content-Type` 确定响应内容的类型（如 HTML、JSON、图片等）。

7. 渲染页面

如果响应内容是 HTML 页面，浏览器会进入渲染阶段：

- 1. **解析 HTML**：浏览器的渲染引擎（如 Blink、Gecko）解析 HTML 文档，构建 **DOM**（文档对象模型）树。
- 2. **解析 CSS**：浏览器解析页面中的 CSS 样式表，构建 **CSSOM**（CSS 对象模型）树。
- 3. **构建渲染树**：将 DOM 树和 CSSOM 树结合，生成渲染树，确定页面的布局和样式。
- 4. **布局（Reflow）**：计算每个元素的位置和大小。
- 5. **绘制（Paint）**：将渲染树转换为屏幕上的像素，显示页面内容。

8. 加载资源

HTML 页面可能包含其他资源（如图片、JavaScript 文件、CSS 文件等）：

- 1. 浏览器会解析 HTML 中的资源链接（如 `<img>`、`<script>`、`<link>`）。
- 2. 浏览器会为每个资源发起新的 HTTP 请求，下载资源。
- 3. 如果是 JavaScript 文件，浏览器会解析并执行脚本。
- 4. 如果是 CSS 文件，浏览器会重新解析样式并更新渲染树。

9. 交互与动态加载

用户与页面交互（如点击按钮、输入内容）可能触发 JavaScript 代码执行：

- 1. JavaScript 可能通过 AJAX 或 Fetch API 发起异步请求，获取新数据。
- 2. 浏览器会根据 JavaScript 的操作动态更新 DOM，重新渲染页面的局部内容。

4.5 前端 job model

P6 单业务执行

模块级优化与问题解决

技术能力

- **技术深度**：熟练使用主流框架（React/Vue），掌握前端工程化（Webpack/Vite），能优化性能、解决复杂业务问题。
- **代码能力**：独立完成复杂模块开发，具备良好的代码架构设计能力。
- **技术广度**：了解 Node.js、微前端等扩展技术，但主要聚焦业务开发。

业务能力

- **执行落地**：负责具体业务模块的开发，能理解产品需求并提出优化建议。
- **协作能力**：带领小型项目，指导 P5 及以下成员，推动团队内部技术改进。

P7 跨业务技术赋能

识别业务痛点，技术驱动效率/增长  
将技术转化为可量化的商业结果  
从“执行者”到“规划者+推动者”

技术能力

- **架构能力**：能设计系统级技术方案，如微服务化、性能优化（如 Weex/HTTPS 升级案例）。
- **前瞻性**：推动新技术落地（如低代码、Serverless），解决公司级技术瓶颈。
- **技术领导力**：主导技术选型，确保架构长期可维护（如“1年后无需重构”标准）。

业务能力

P8 战略级业务驱动

精通团队相关技术业务：关注业务的整体情况管理

技术能力

- **战略级架构**：主导公司级前端基建（如统一工程化体系、Flutter/Node 技术布局）。
- **行业影响力**：参与开源项目、技术大会分享，推动行业标准（如 Rax 体系）。
- **技术创新**：探索前沿技术（如 PWA、Service Worker），并判断其业务适用性。

业务能力

- **业务战略**：参与业务顶层设计，技术驱动业务增长（如 10 亿+ HTTP 业务 HTTPS 化）。

- **影响力：**主要在团队内部，较少跨部门协作。
- **业务规划：**深度参与业务决策，识别技术对业务的杠杆作用（如提升秒开率 92-95%）。
- **跨团队协作：**推动跨部门项目（如前端与客户端协作 Weex 化），影响业务技术方向。
- **团队管理：**通常带 4-5 人小团队，部分 P7 开始承担 TL（Team Leader）职责。
- **团队规模：**管理 10-30 人团队，负责 HC 分配、绩效考核。
- **商业价值：**技术方案直接影响公司核心指标（如降低运维成本、提升开发效率）。